Java w IntelliJ IDEA - zmienne, typy danych i operatory

BLOK 1 - zmienne i typy danych

1. Przypomnienie - najprostszy program

Otwórz w pakiecie block1 klasę Example01Hello.

```
package block1;

public class Example01Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java!");
    }
}
```

Uruchom program zielonym przyciskiem obok metody main i wybierz Run 'Example01Hello.main()'.

Wynik:

```
Hello Java!
```

Na tym etapie program wykonuje zawsze dokładnie to samo. Tekst „Hello Java!" jest wpisany na stałe w kodzie. Za chwilę zobaczymy, jak przechowywać dane w zmiennych, aby można było z nich wielokrotnie korzystać i je zmieniać.

2. Czym jest zmienna?

Program pracuje na danych. Danymi mogą być na przykład: wiek użytkownika, cena produktu, liczba punktów w grze, imię studenta albo informacja, czy konto jest aktywne.

Jeżeli wartość wpiszemy bezpośrednio do instrukcji, program może jej użyć tylko w tym konkretnym miejscu:

```
System.out.println(20);
```

W większym programie ta sama informacja może być potrzebna w wielu miejscach. Dlatego nadajemy jej nazwę i przechowujemy ją w zmiennej.

Zmienna to nazwane miejsce, w którym program przechowuje wartość określonego typu. Dzięki nazwie możemy później odwoływać się do tej wartości, wyświetlać ją, wykonywać na niej obliczenia albo zastąpić ją inną wartością.

Można myśleć o zmiennej jak o podpisanym pudełku:

```
age -> 20
points -> 150
name -> "Anna"
```

Nazwa mówi, czego dotyczy dana wartość, a typ określa, jakiego rodzaju dane mogą zostać w tym miejscu zapisane.

3. Po co używamy zmiennych?

Zmienne są potrzebne, ponieważ większość programów nie operuje cały czas na tych samych wartościach. Dane mogą się zmieniać podczas działania programu.

Przykład bez zmiennej:

```
System.out.println("Punkty gracza: 100");
System.out.println("Punkty po bonusie: 150");
```

Tutaj wartości 100 i 150 zostały wpisane ręcznie. Program nie oblicza drugiej wartości na podstawie pierwszej.

Przykład ze zmienną:

```
int points = 100;
System.out.println("Punkty gracza: " + points);

points = points + 50;
System.out.println("Punkty po bonusie: " + points);
```

Teraz wartość jest przechowywana w jednym miejscu, może być zmieniana i wykorzystywana wiele razy.

Zmienne pozwalają więc:

- zapamiętywać dane,
- nadawać danym czytelne nazwy,
- używać tej samej wartości w wielu miejscach programu,
- zmieniać wartość w trakcie działania programu,
- wykonywać obliczenia na danych,
- budować programy, które później mogą pracować na danych podawanych przez użytkownika, odczytywanych z pliku lub pobieranych z innych źródeł.

4. Budowa deklaracji zmiennej

Najprostsza deklaracja z jednoczesnym przypisaniem wartości wygląda tak:

```
int age = 20;
```

Elementy tej instrukcji:

Element	Znaczenie	Przykład
typ	określa rodzaj danych	int
nazwa	pozwala odwoływać się do wartości	age
wartość	dane zapisane w zmiennej	20

Średnik ; kończy instrukcję.

5. Deklaracja, inicjalizacja i przypisanie

Warto rozróżniać trzy pojęcia:

- deklaracja - utworzenie zmiennej i podanie jej typu oraz nazwy,
- inicjalizacja - nadanie wartości początkowej zmiennej w momencie jej deklaracji,
- przypisanie - nadanie lub zmiana wartości już zadeklarowanej zmiennej,

Deklaracja bez wartości:

```
int age;
```

Przypisanie pierwszej wartości:

```
age = 20;
```

Można też wykonać oba kroki jednocześnie:

```
int age = 20;
```

Późniejsza zmiana wartości:

```
age = 21;
```

Nie piszemy ponownie int przy każdej zmianie wartości. Typ podajemy podczas deklaracji zmiennej. Później używamy już samej nazwy.

Przykład 1. Pierwsza zmienna

```
package block1;

public class Example02Variables {
    public static void main(String[] args) {
        int age = 20;

        System.out.println(age);
    }
}
```

Wynik:

```
20
```

Program nie wyświetla słowa age. Java pobiera wartość zapisaną w zmiennej age i przekazuje ją do System.out.println().

Przykład 2. Zmiana wartości zmiennej

```
package block1;

public class Example03ChangeValue {
    public static void main(String[] args) {
        int points = 10;

        System.out.println(points);

        points = 20;

        System.out.println(points);
    }
}
```

Wynik:

```
10
20
```

Zmienna points nadal jest tą samą zmienną. Zmieniła się tylko przechowywana w niej wartość.

6. Nazwy zmiennych

Dobra nazwa zmiennej powinna informować, co przechowuje dana zmienna.

Lepsze nazwy:

```
int age = 20;
double productPrice = 49.99;
String studentName = "Anna";
```

Mało czytelne nazwy:

```
int a = 20;
double x = 49.99;
String s = "Anna";
```

Podstawowe zasady:

- nazwa nie może zawierać spacji,
- nie może zaczynać się od cyfry,
- nazwa nie może być słowem kluczowym języka Java, np. `int`, `class`, `public`
- Java rozróżnia wielkie i małe litery, więc age i Age to różne nazwy,
- w typowym kodzie Java nazwy zmiennych zapisujemy stylem camelCase, np. productPrice, studentName, numberOfStudents,
- należy unikać nazw, które nic nie mówią o znaczeniu danych.

```
int class = 5;       // błąd

int classNumber = 5; // poprawnie
```

7. Co oznacza typ zmiennej?

Typ danych określa, jakiego rodzaju wartość może przechowywać zmienna i jakie operacje można na niej wykonywać.

Przykład:

```
int age = 20;
```

Zmienna age ma typ int, więc przechowuje liczbę całkowitą. Nie można później przypisać do niej tekstu:

```
age = "dwadzieścia"; // błąd
```

Java jest językiem statycznie typowanym. Typ zmiennej jest sprawdzany podczas kompilacji. IntelliJ IDEA bardzo często pokaże taki problem jeszcze przed uruchomieniem programu.

Dobór typu ma znaczenie. Program powinien używać typu odpowiadającego rzeczywistemu znaczeniu danych.

8. Podstawowe typy, których będziemy używać

Typ	Przechowuje	Przykład	Kiedy używać
int	liczbę całkowitą	20	wiek, liczba sztuk, punkty
double	liczbę z częścią ułamkową	19.99	cena, wzrost, temperatura
char	jeden znak	'A'	litera grupy, pojedynczy symbol
boolean	prawdę lub fałsz	true	stan: aktywny/nieaktywny, tak/nie

String	tekst	"Anna"	imię, nazwa produktu, komunikat

Ważne: `int`, `double`, `char` i `boolean` należą do typów prostych (`primitive types`). `String` nie jest typem prostym — jest klasą, czyli typem referencyjnym. Na tym etapie będziemy używać `String` po prostu do przechowywania tekstu.

9. Typ int - liczby całkowite

Typ int wykorzystujemy wtedy, gdy przechowywana wartość jest liczbą całkowitą i nie potrzebujemy części ułamkowej.

```
int age = 20;
int students = 25;
int temperature = -5;
int points = 0;
```

Typ int pasuje np. do liczby studentów, liczby punktów lub liczby sztuk produktu.

Nie pasuje do wartości 1.75, ponieważ zawiera ona część ułamkową.

```
int height = 1.75; // błąd
```

Przykład 3. Kilka zmiennych typu int

```
package block1;

public class Example04Int {
   public static void main(String[] args) {
      int age = 21;
      int points = 120;
      int temperature = -3;

      System.out.println(age);
      System.out.println(points);
      System.out.println(temperature);
   }
}
```

10. Typ double - liczby z częścią ułamkową

Typ double wykorzystujemy do liczb, które mogą zawierać część ułamkową.

```
double price = 19.99;
double height = 1.82;
double temperature = 21.5;
```

W kodzie Java separatorem dziesiętnym jest kropka, niezależnie od polskiego sposobu zapisu liczb.

Poprawnie:

```
double price = 12.50;
```

Niepoprawnie:

```
double price = 12,50;
```

Liczbę całkowitą można przypisać do double:

```
double price = 10;
```

Java potraktuje ją jako wartość zmiennoprzecinkową. W drugą stronę nie działa to automatycznie:

```
int price = 10.5; // błąd
```

Uwaga: w tych przykładach wykorzystujemy double do prostych obliczeń z cenami. W rzeczywistych aplikacjach finansowych, gdy wymagana jest dokładność dziesiętna, często stosuje się `BigDecimal`.

Przykład 4. Typ double

```
package block1;

public class Example05Double {
   public static void main(String[] args) {
      double price = 129.99;
      double height = 1.78;

      System.out.println(price);
      System.out.println(height);
   }
}
```

11. Typ char - pojedynczy znak

Na tym etapie będziemy traktować char jako typ przechowujący pojedynczy znak, np. 'A', `'7'`, `'#'`.

```
char group = 'A';
char grade = 'B';
char symbol = '#';
```

Wartość char zapisujemy w pojedynczych apostrofach.

Poprawnie:

```
char letter = 'J';
```

Niepoprawnie:

```
char letter = "J";
```

Drugi zapis używa podwójnych cudzysłowów, więc reprezentuje String, a nie char.

Nie można też umieścić kilku znaków w char:

```
char text = 'Java'; // błąd
```

12. Typ boolean - prawda albo fałsz

Typ boolean przechowuje wartość logiczną. Może mieć tylko dwie wartości: true albo false.

```
boolean student = true;
boolean gameOver = false;
boolean active = true;
```

Typ `boolean` jest przydatny do opisywania stanów, które mają dwie możliwości, np.:

- konto aktywne / nieaktywne,
- gracz żyje / nie żyje,
- użytkownik jest zalogowany / niezalogowany,
- student zaliczył / nie zaliczył.

Na kolejnych zajęciach wartości boolean będą wykorzystywane w instrukcjach warunkowych if.

true i false nie są tekstem. Nie zapisujemy ich w cudzysłowach.

```
boolean active = true;      // poprawnie
boolean active2 = "true";   // błąd
```

13. Typ String - tekst

String służy do przechowywania ciągu znaków, czyli tekstu.

```
String name = "Anna";
String city = "Wrocław";
String language = "Java";
```

Wartość String zapisujemy w podwójnych cudzysłowach.

String może zawierać jeden znak, wiele znaków, spacje, cyfry oraz symbole:

```
String letter = "A";
String sentence = "Programowanie w Javie";
String postalCode = "53-609";
```

Warto zauważyć, że "123" jest tekstem, natomiast 123 jest liczbą. Wyglądają podobnie, ale program traktuje je inaczej.

14. char a String - jaka jest różnica?

char przechowuje jeden znak i używa apostrofów. String przechowuje tekst i używa cudzysłowów.

```
char group = 'A';
String name = "Anna";
String oneLetter = "A";
```

Wartość "A" jest Stringiem, nawet jeśli zawiera tylko jeden znak.

15. Łączenie tekstu z wartościami zmiennych

Operator + może służyć do łączenia tekstu z wartością zmiennej.

```
String name = "Anna";
int age = 20;

System.out.println("Imię: " + name);
System.out.println("Wiek: " + age);
```

Wynik:

```
Imię: Anna
Wiek: 20
```

Gdy jednym z argumentów operatora + jest String, Java wykonuje konkatenację tekstu, konwertując drugi operand do reprezentacji tekstowej.

Przykład 5. Dane studenta

```
package block1;

public class Example06Student {
    public static void main(String[] args) {
        String name = "Anna";
        int age = 20;
        double average = 4.25;
        char group = 'A';
```

```
        boolean student = true;

        System.out.println("Imię: " + name);
        System.out.println("Wiek: " + age);
        System.out.println("Średnia: " + average);
        System.out.println("Grupa: " + group);
        System.out.println("Student: " + student);
    }
}
```

Wynik:

```
Imię: Anna
Wiek: 20
Średnia: 4.25
Grupa: A
Student: true
```

Każda informacja ma tutaj typ dopasowany do jej znaczenia: imię jest tekstem, wiek liczbą całkowitą, średnia może mieć część ułamkową, grupa to pojedynczy znak, a informacja student/nie student jest wartością logiczną.

16. Najczęstsze błędy przy zmiennych

1. Użycie tekstu bez cudzysłowów:

```
String name = Anna; // błąd
```

2. Próba zapisania tekstu w int:

```
int age = "20"; // błąd
```

3. Próba zapisania liczby ułamkowej w int:

```
int height = 1.80; // błąd
```

4. Użycie podwójnych cudzysłowów dla char:

```
char group = "A"; // błąd
```

5. Użycie cudzysłowów dla boolean:

```
boolean active = "true"; // błąd
```

6. Użycie zmiennej przed przypisaniem wartości:

```
int age;
System.out.println(age); // błąd: brak wartości
```

17. Zadania - blok 1

Zadanie 1. MyData

Otwórz klasę MyData w pakiecie block1.

Utwórz zmienne przechowujące imię, wiek, wzrost, literę grupy oraz informację, czy jesteś studentem.

Dobierz typ każdej zmiennej i zastanów się, dlaczego właśnie ten typ pasuje do danej informacji.

Wyświetl wszystkie wartości w czytelnej postaci.

Zadanie 2. Computer

Otwórz klasę Computer.

Przechowaj w zmiennych producenta procesora, liczbę rdzeni, ilość pamięci RAM, cenę oraz informację, czy komputer jest włączony.

Nie wpisuj wartości bezpośrednio do System.out.println() - użyj utworzonych zmiennych.

Przykładowy wynik:

```
Procesor: Intel
Rdzenie: 8
RAM: 16 GB
Cena: 3499.99 zł
Włączony: true
```

Zadanie 3. Znajdź błędy

Wklej poniższy kod do osobnej klasy.

Dla każdego błędu spróbuj najpierw wyjaśnić, dlaczego podana wartość nie pasuje do typu zmiennej.

Poniższy kod celowo zawiera błędy i nie powinien się skompilować.

Następnie popraw kod i uruchom program.

```
public class Errors {
    public static void main(String[] args) {
        String name = Jan;
        int age = "20";
        double height = 1,80;
        char group = "A";
        boolean student = "true";

        System.out.println(name);
    }
}
```

Zadanie 4. Dobierz typ

Otwórz klasę TypeSelection w pakiecie block1.

Dla każdej informacji zaproponuj odpowiedni typ Java: wiek studenta, imię studenta, cena biletu, litera grupy, liczba punktów, informacja czy konto jest aktywne, temperatura 21.5°C.

Następnie utwórz odpowiednie zmienne w kodzie.

BLOK 2 - operatory i zmiana wartości

18. Dlaczego wykonujemy działania na zmiennych?

Samo przechowywanie danych to dopiero początek. Program często musi obliczyć nową wartość na podstawie danych już zapisanych w zmiennych.

Przykład: znając cenę jednej sztuki i liczbę sztuk, możemy obliczyć całkowitą cenę zakupu.

```
double price = 12.50;
int quantity = 4;
double total = price * quantity;
```

Wynik nie jest wpisany ręcznie. Powstaje na podstawie aktualnych wartości zmiennych.

19. Operatory arytmetyczne

Operator	Znaczenie	Przykład
+	dodawanie	a + b
-	odejmowanie	a - b
*	mnożenie	a * b
/	dzielenie	a / b
%	reszta z dzielenia	a % b

Przykład 6. Proste obliczenia

```
package block2;

public class Example07Arithmetic {
   public static void main(String[] args) {
      int a = 20;
      int b = 4;

      int sum = a + b;
      int difference = a - b;
      int product = a * b;
      int quotient = a / b;

      System.out.println("Suma: " + sum);
      System.out.println("Różnica: " + difference);
      System.out.println("Iloczyn: " + product);
      System.out.println("Iloraz: " + quotient);
   }
}
```

Wynik:

```
Suma: 24
Różnica: 16
Iloczyn: 80
Iloraz: 5
```

20. Dzielenie typu int

Jeżeli oba argumenty są typu całkowitoliczbowego, wykonywane jest dzielenie całkowitoliczbowe. Wynik jest zaokrąglany w kierunku zera, dlatego `10 / 3` daje `3`.

```
int result = 10 / 3;
System.out.println(result);
```

Wynik:

```
3
```

Jeżeli zależy nam na wyniku z częścią ułamkową, co najmniej jedna wartość powinna być typu zmiennoprzecinkowego:

```
double result = 10.0 / 3.0;
System.out.println(result);
```

21. Operator % - reszta z dzielenia

Operator % nie zwraca wyniku dzielenia, lecz resztę, która zostaje po wykonaniu dzielenia całkowitego.

```
10 % 3  // 1
10 % 2  // 0
11 % 2  // 1
```

Ten operator będzie później przydatny np. do sprawdzania, czy liczba jest parzysta.

Przykład 7. Reszta z dzielenia

```
package block2;

public class Example08Modulo {
    public static void main(String[] args) {
        System.out.println(10 % 3);
        System.out.println(10 % 2);
        System.out.println(11 % 2);
    }
}
```

Wynik:

```
1
0
1
```

22. Kolejność wykonywania działań

Java stosuje standardową kolejność działań. Mnożenie, dzielenie i operator % są wykonywane przed dodawaniem i odejmowaniem.

```
int result = 2 + 3 * 4;
System.out.println(result);
```

Wynik:

```
14
```

Nawiasy zmieniają kolejność:

```
int result = (2 + 3) * 4;
System.out.println(result);
```

Wynik:

```
20
```

23. Obliczenia na podstawie zmiennych

Przykład 8. Pole i obwód prostokąta

```
package block2;

public class Example09Rectangle {
   public static void main(String[] args) {
      double width = 10.0;
      double height = 5.0;

      double area = width * height;
      double perimeter = 2 * width + 2 * height;

      System.out.println("Pole: " + area);
      System.out.println("Obwód: " + perimeter);
   }
}
```

Wynik:

```
Pole: 50.0
Obwód: 30.0
```

Jeśli zmienimy width albo height, nie musimy ręcznie zmieniać wyniku. Program obliczy go ponownie na podstawie nowych wartości.

24. Zmiana wartości na podstawie poprzedniej wartości

```
int points = 100;
points = points + 50;
```

Najpierw Java odczytuje aktualną wartość points, czyli 100, dodaje 50, a dopiero potem zapisuje wynik 150 z powrotem do points.

Znak = w Javie oznacza przypisanie. Nie jest tutaj znakiem równości znanym z matematyki.

25. Skrócone operatory przypisania

Operacje typu „weź aktualną wartość i dodaj coś” występują bardzo często, dlatego Java ma skrócone operatory.

```
points = points + 50;
// to samo co:
points += 50;
```

W tym przypadku instrukcję `points = points + 50;` możemy zapisać krócej jako `points += 50;`.

Analogicznie:

```
points -= 20;
points *= 2;
points /= 2;
```

Przykład 9. Zmiana punktów

```
package block2;

public class Example10Assignment {
   public static void main(String[] args) {
      int points = 100;
      System.out.println(points);

      points += 50;
      System.out.println(points);

      points -= 20;
      System.out.println(points);

      points *= 2;
      System.out.println(points);
   }
}
```

Wynik:

```
100
150
130
260
```

26. Inkrementacja i dekrementacja

Jeżeli chcemy zwiększyć wartość dokładnie o 1, możemy użyć operatora ++.

```
counter++;
// odpowiednik:
counter = counter + 1;
```

Operator -- zmniejsza wartość o 1:

```
counter--;
// odpowiednik:
counter = counter - 1;
```

Przykład 10. Licznik

```
package block2;

public class Example11Counter {
   public static void main(String[] args) {
      int counter = 0;

      System.out.println(counter);
      counter++;
      System.out.println(counter);
      counter++;
      System.out.println(counter);
      counter--;
      System.out.println(counter);
   }
}
```

Wynik:

```
0
1
2
1
```

27. number++ i ++number

Otwórz w pakiecie block2 klasę Example12IncrementOrder i wykorzystaj ją do porównania postinkrementacji i preinkrementacji.

Gdy ++ jest osobną instrukcją, oba zapisy zwiększają wartość o 1:

```
number++;
++number;
```

Różnica jest widoczna wtedy, gdy operator jest częścią większego wyrażenia.

Postinkrementacja: najpierw używana jest stara wartość, potem następuje zwiększenie.

```
int number = 5;
System.out.println(number++);
System.out.println(number);
```

Wynik:

```
5
6
```

Preinkrementacja: najpierw następuje zwiększenie, potem używana jest nowa wartość.

```
int number = 5;
System.out.println(++number);
System.out.println(number);
```

Wynik:

```
6
6
```

Na tym etapie najczytelniej używać ++ i -- jako osobnych instrukcji.

28. Liczba czy tekst?

Otwórz w pakiecie block2 klasę Example13StringAndNumbers i sprawdź, jak operator + zachowuje się dla liczb i tekstu.

Operator + może oznaczać dodawanie liczb albo łączenie tekstu. Znaczenie zależy od typów danych.

```
System.out.println(2 + 3);
System.out.println("2" + "3");
System.out.println("Wynik: " + 2 + 3);
System.out.println("Wynik: " + (2 + 3));
```

Wynik:

```
5
23
Wynik: 23
Wynik: 5
```

W trzecim przykładzie Java rozpoczyna od tekstu, więc kolejne wartości również dołącza do tekstu. Nawiasy w czwartym przykładzie wymuszają wcześniejsze wykonanie dodawania liczb.

29. Zadania - blok 2

Zadanie 5. Calculator

Otwórz klasę Calculator.

Dla zmiennych a = 20 i b = 4 oblicz sumę, różnicę, iloczyn, iloraz i resztę z dzielenia.

Każdy wynik zapisz w osobnej zmiennej.

```
int a = 20;
int b = 4;
```

Zadanie 6. Przewidź wynik

Otwórz klasę PredictResult w pakiecie block2.

Najpierw zapisz przewidywane wyniki bez uruchamiania programu.

Następnie sprawdź je w IntelliJ IDEA i wyjaśnij ewentualne różnice.

```
System.out.println(20 % 3);
System.out.println(20 % 5);
System.out.println(13 % 2);
System.out.println(12 / 5);
System.out.println(12.0 / 5.0);
```

Zadanie 7. Śledzenie wartości zmiennej

Otwórz klasę TrackValue w pakiecie block2.

Uzupełnij wartość x po każdej instrukcji. Dopiero później uruchom kod.

```
int x = 10;
x += 5;
x++;
x *= 2;
x -= 4;
x--;
```

Zadanie 8. Shopping

Otwórz klasę Shopping.

Oblicz całkowity koszt zakupu na podstawie ceny jednej sztuki i liczby sztuk.

```
String product = "Klawiatura";
double price = 129.99;
int quantity = 2;

double total = ...;
```

Przykładowy wynik:

```
Produkt: Klawiatura
Cena: 129.99
Liczba sztuk: 2
Razem: 259.98
```

Zadanie 9. Wallet

Otwórz klasę Wallet.

Stan początkowy to 100.0 zł.

Następnie wykonaj: +50 zł, -20 zł, -15.50 zł, +30 zł.

Wykorzystaj += oraz -= i po każdej zmianie wyświetl stan portfela.

```
double money = 100.0;
```

Zadanie 10. Game

Otwórz klasę Game.

Wykorzystaj +=, ++ oraz --, aby zmodyfikować dane zgodnie ze scenariuszem.

```
String player = "Player1";
int level = 1;
int points = 0;
int lives = 3;

// +100 punktów
// +50 punktów
// +1 poziom
// -1 życie
// +200 punktów
```

Przykładowy wynik:

```
Gracz: Player1
Poziom: 2
Punkty: 350
Życia: 2
```

Zadanie 11. Time

Dla podanej liczby godzin oblicz liczbę minut i sekund.

Nie wpisuj gotowych wyników - program ma je obliczyć.

```
int hours = 3;
// 1 godzina = 60 minut
// 1 minuta = 60 sekund
```

Zadanie 12. Temperature

Oblicz temperaturę w stopniach Fahrenheita według wzoru F = C * 9 / 5 + 32.

Zwróć uwagę, że `celsius` jest typu `double`. Dzięki temu całe działanie jest wykonywane jako obliczenie zmiennoprzecinkowe. Samo `9 / 5` wykonane na liczbach `int` dałoby wynik `1`.

```
double celsius = 25.0;
```

Zadanie 13. Znajdź błędy

Poniższy kod celowo zawiera błędy i nie powinien się skompilować.

Popraw wszystkie błędy wskazane przez IntelliJ IDEA.

Przy każdym błędzie spróbuj nazwać jego przyczynę: zły typ, brak średnika, brak operatora + itd.

```
public class Shop {
    public static void main(String[] args) {
        String product = "Monitor"
        double price = "899.99";
        int quantity = 2;

        double total = price * quantity;

        System.out.println("Produkt: " + product);
        System.out.println("Cena: " price);
```

```
        System.out.println("Liczba: " + quantity);
        System.out.println("Razem: " + total)
    }
}
```

Zadanie 14. Mystery

Nie uruchamiaj programu od razu.

Najpierw przewidź trzy wartości, a potem sprawdź je w IntelliJ IDEA.

```
int a = 5;
int b = 2;
int c = a + b * 3;
a++;
b += 4;
System.out.println(a);
System.out.println(b);
System.out.println(c);
```

30. Zadania dodatkowe

Zadanie dodatkowe A. Kilometry

Otwórz klasę Kilometers w pakiecie block2.

Dla wartości 7.5 km oblicz liczbę metrów.

```
double kilometers = 7.5;
```

Zadanie dodatkowe B. Czas

Otwórz klasę DaysTime w pakiecie block2.

Dla 7 dni oblicz liczbę godzin, minut i sekund.

```
int days = 7;
```

Zadanie dodatkowe C. Zamówienie

Otwórz klasę Order w pakiecie block2.

Oblicz koszt pizz, koszt napojów oraz całkowitą wartość zamówienia.

```
double pizzaPrice = 32.50;
int pizzas = 3;
double drinkPrice = 8.00;
int drinks = 4;
```